

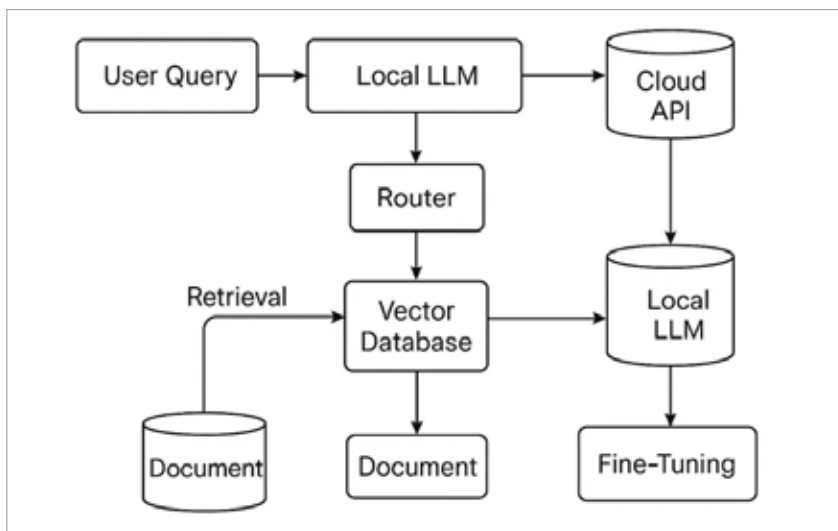


Figure 1: Sovereign AI

As a result, hardware selection plays a crucial role in determining both performance and usability. Unlike cloud platforms that abstract infrastructure concerns, local deployment requires a practical understanding of how CPUs, GPUs, memory, and storage interact during model inference. The good news is that modern consumer hardware is far more capable than it appears, provided it is used efficiently.

For many entry-level setups, the central processing unit serves as the starting point. Contemporary multi-core CPUs can run smaller language models reasonably well, especially when paired with optimised inference engines. While CPU-based inference may not match the responsiveness of GPU-backed systems, it remains a viable option for experimentation, learning, and lightweight assistants. This makes local AI accessible even on machines without dedicated graphics hardware.

Graphics processing units significantly enhance the experience of running local LLMs. GPUs accelerate matrix operations that are fundamental to deep learning inference, resulting in faster response times and smoother interactions. Video memory becomes a key consideration here, as larger models require more VRAM to load efficiently.

Techniques such as model quantization allow these models to consume less memory, enabling them to run on mid-range consumer GPUs without compromising usability.

Beyond compute hardware, system memory and storage also influence performance. Adequate RAM ensures that models load smoothly and prevents bottlenecks during execution, while fast

solid-state storage reduces initialisation time. Equally important is thermal management, as sustained inference workloads can push hardware to its limits. With thoughtful optimisation and realistic expectations, even modest systems can serve as capable platforms for private AI assistants.

The software stack: Choosing your engine

Once the hardware foundation is in place, the real personality of a private AI assistant emerges from the software stack that powers it. Unlike cloud-based systems where infrastructure and tooling are hidden behind APIs, local deployments require conscious choices at every layer. The inference engine, model format, and runtime environment together determine how efficiently a language model runs and how flexible the system becomes over time.

At the core of the stack lies the inference engine, which is responsible for loading the model and executing predictions. Modern open source engines are designed to squeeze maximum performance from available

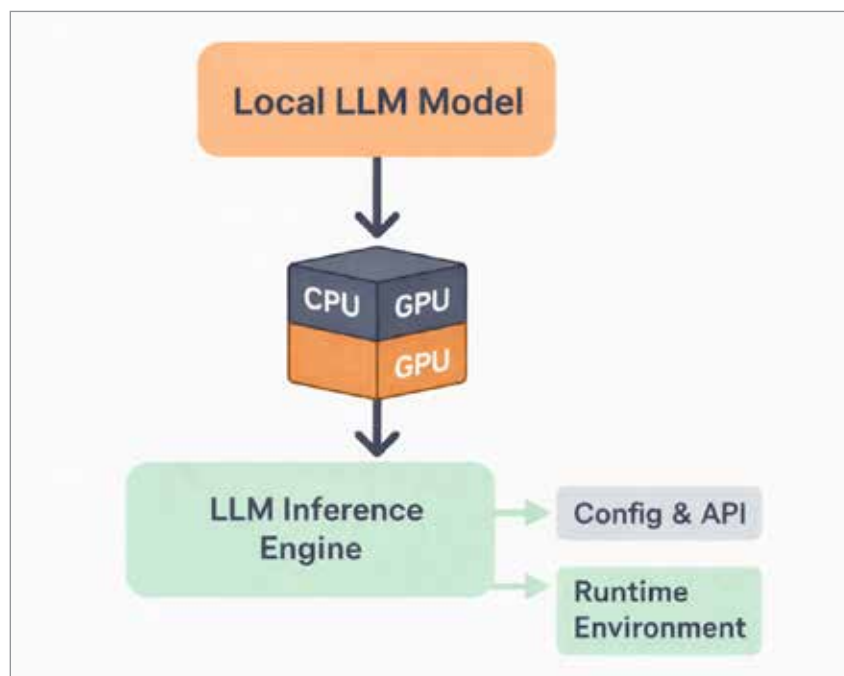


Figure 2: The architecture